

Acid and indexes

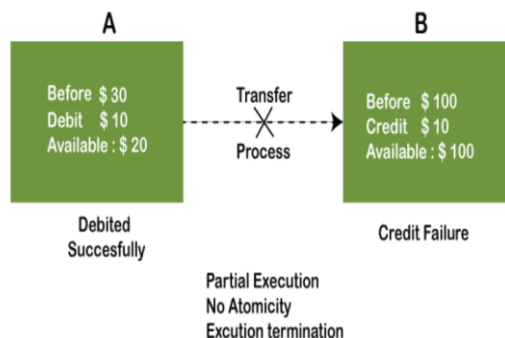
ACID refers to a set of properties that ensure reliable processing of database transactions. ACID stands for **A**tomicity, **C**onsistency, **I**solation, and **D**urability. These properties guarantee that database transactions are processed reliably and that the database remains in a consistent state, even in the event of errors, power failures, or other unexpected issues.

Types of acid:

1. Atomicity
2. Consistency
3. Isolation
4. Durability

1. Atomicity:

Definition: Atomicity ensures that a transaction is treated as a single unit of operation. This means that either all the operations within the transaction are executed, or none of them are.



Example in MongoDB: If a transaction involves multiple write operations, MongoDB ensures that either all writes are completed successfully or none are applied.

Benefits of Atomicity in MongoDB:

- **Ensures Data Consistency:** The primary benefit of atomicity is that it prevents documents from ending up in an inconsistent state. Updates to a single document are treated as a whole, meaning either all changes are applied successfully or none are.
- **Prevents Data Corruption:** By guaranteeing all or nothing updates, atomicity safeguards against data corruption that might occur if only a portion of an intended change is applied.
- **Improves Data Integrity:** Since atomicity ensures complete or no updates, it strengthens the overall integrity of your data.
- **Provides a Reliable Update Mechanism:** Atomicity simplifies data manipulation by providing a reliable all-or-nothing approach..

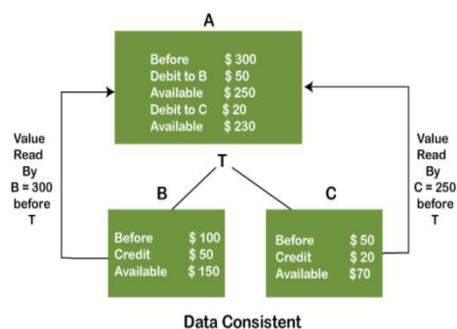
- **Reduces Need for Complex Error Handling:** With atomicity, the need for intricate error handling code to address partial updates is lessened. If an error occurs during an operation, the entire update is rolled back, leaving the document unchanged.

2.Consistency:

Definition: Consistency ensures that a transaction brings the database from one valid state to another, maintaining all predefined rules such as constraints and triggers.

The word **consistency** means that the value should remain preserved always. In DBMS, the integrity of the data should be maintained, which means if a change in the database is made, it should remain preserved always. In the case of transactions, the integrity of the data is very essential so that the database remains consistent before and after the transaction. The data should always be correct.

Example:



Types of Consistency in MongoDB:

1. Eventual Consistency (Default):

- This is the default consistency model in MongoDB. Data updates are replicated asynchronously across the cluster. While eventually all replicas will have the same data, there might be a brief lag between when a write operation is committed on the primary node and when it's reflected on all secondary nodes.

2. Strong Consistency (Read After Write):

- This consistency model ensures that after a write operation is committed on the primary node, any subsequent read operation (even on a secondary) will always return the latest data.
- This is achieved by forcing the read operation to wait until confirmation is received from a configurable number of secondary nodes acknowledging the write.

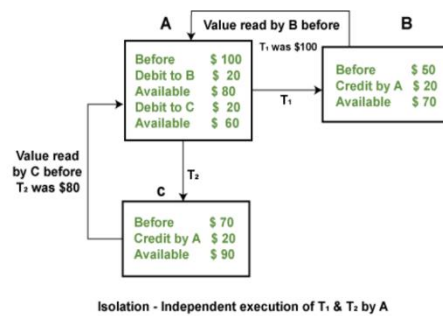
3. Causal Consistency (Client Sessions):

- This consistency model focuses on ensuring a specific order for causally related operations within a client session.
- This requires enabling causally consistent client sessions and using specific read and write concerns within the session.

3.Isolation:

Definition: Isolation ensures that the operations in a transaction are isolated from other concurrent transactions. This means that intermediate states of a transaction are not visible to other transactions.

Example: If two operations are concurrently running on two different accounts, then the value of both accounts should not get affected. The value should remain persistent. As you can see in the below diagram, account A is making T1 and T2 transactions to account B and C, but both are executing independently without affecting each other. It is known as Isolation.



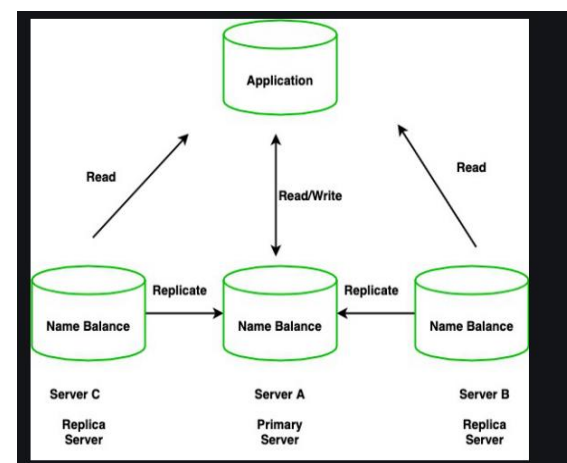
4. Durability

Definition: Durability guarantees that once a transaction has been committed, it will remain so, even in the event of a system crash or power failure.

Example in MongoDB: MongoDB writes operations to the journal, ensuring that committed transactions are durable and can be recovered in case of a crash.

Replication (Master - Slave):

Replication in MongoDB is a process that involves creating multiple copies of data across different servers to ensure high availability, data redundancy, and fault tolerance. This allows your MongoDB database to remain operational even in the event of hardware failures, server crashes, or maintenance tasks.



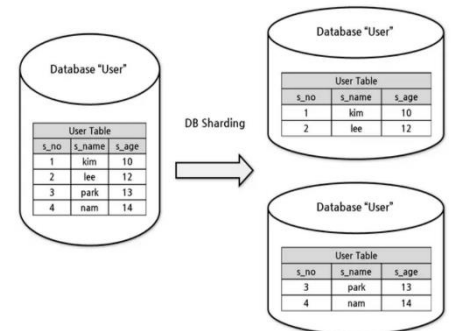
Benefits of Replication:

- **High Availability:** Automatic failover to a secondary node ensures minimal downtime.
- **Data Redundancy:** Multiple copies of data protect against data loss.
- **Horizontal Scaling:** Enables distributing read operations across multiple nodes.

- **Disaster Recovery:** Provides data backup and recovery option

Sharding:

Sharding in MongoDB is a method for distributing data across multiple machines to support deployments with large data sets and high throughput operations. This technique enables MongoDB to handle more data than a single server can store or manage and improves performance by distributing the load.



Key Concepts of Sharding

- 1.Shard
2. Config Servers
- 3.Query Router (Mongos)
- 4.Sharded Key

How Sharding Works

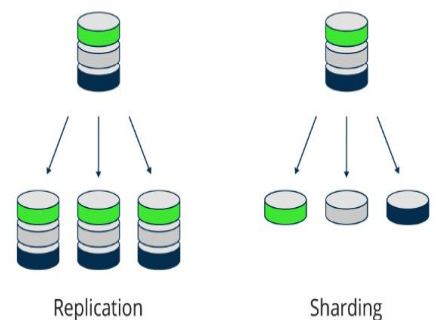
1. Data Distribution
- 2.Balancing
3. Routing Queries
4. Scalability

Benefits of Sharding

- Horizontal Scalability
- Improved Performance
- High Availability

Replication VS Sharding:

Replication and sharding are two critical components of MongoDB's architecture, each serving distinct purposes. Understanding the differences between replication and sharding helps in designing scalable, reliable, and high-performance MongoDB deployments.



Replication in MongoDB

Purpose

- **High Availability:** Replication provides data redundancy by maintaining multiple copies of the data across different servers.
- **Fault Tolerance:** It ensures that if one server fails, another can take over without data loss.
- **Read Scalability:** Enables read operations on secondary nodes, reducing the load on the primary node.

Benefits

- **Data Redundancy:** Multiple copies of data protect against data loss.
- **High Availability:** Automatic failover ensures continuous operation.
- **Read Performance:** Secondary nodes can handle read operations, reducing the load on the primary.

Sharding in MongoDB

Purpose

- **Horizontal Scalability:** Sharding allows MongoDB to distribute large datasets across multiple servers, providing horizontal scaling.
- **Write Scalability:** It improves write performance by distributing writes across multiple shards.

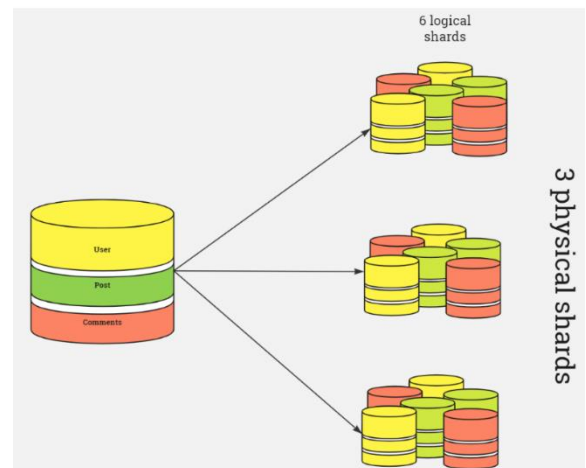
Benefits

- **Horizontal Scalability:** Distributes data across multiple nodes, allowing for growth in storage and processing power.
- **Improved Write Performance:** Distributes write operations across shards, reducing the load on individual nodes.
- **Data Distribution:** Enables efficient management of large datasets by partitioning data.

Aspect	Replication	Sharding
Purpose	High availability and fault tolerance	Horizontal scalability and data distribution
Data Distribution	Copies entire dataset across replica set members	Distributes data across shards based on a shard key
Scalability	Improves read scalability	Improves both read and write scalability
Components	Primary and secondary nodes in a replica set	Shards, config servers, and mongos
Failover	Automatic failover to a secondary node	No automatic failover, but redundancy can be achieved using replica sets for shards
Complexity	Simpler setup and management	More complex due to data partitioning and routing
Use Cases	Disaster recovery, read-heavy workloads, backup solutions	Large datasets, write-heavy applications, geographically distributed systems
Read Consistency	Consistency depends on read preferences and write concerns	Consistency determined by shard key and data distribution strategy
Data Redundancy	Provides redundancy by replicating data across multiple nodes	No inherent redundancy; redundancy achieved by using replica sets within shards

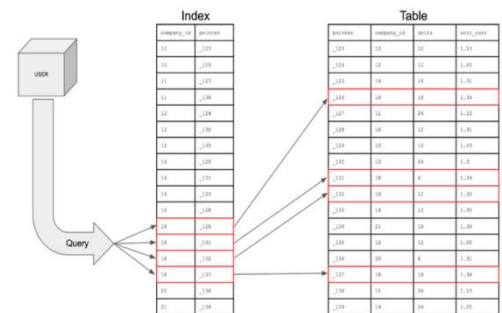
Replication + Sharding

Replication and sharding are powerful features of MongoDB, each addressing specific needs for data management. While replication focuses on high availability and fault tolerance, sharding emphasizes horizontal scalability and data distribution. Combining both techniques allows MongoDB to handle large-scale applications with high performance, reliability, and efficiency. When designing a MongoDB deployment, carefully consider the application's needs to determine the appropriate use of replication, sharding, or both.



Indexing in MongoDB:

Indexing in MongoDB is a powerful feature that enhances the performance and efficiency of database operations. By creating indexes on fields, MongoDB can quickly locate and access data, significantly reducing the time required for read and write operations. Here's a comprehensive overview of indexing in MongoDB, including its types, benefits, and best practices.



Types of Indexes in MongoDB

1.Single Field Indexes:

- **Description:** An index on a single field in a document.
- **Use Case:** Suitable for queries that filter on a single field.
- **Example:**

```
db.collection.createIndex({ fieldName: 1 });
```

- 1 specifies ascending order. Use -1 for descending order.

2.Compound Indexes :

- **Description:** Indexes on multiple fields within a document.
- **Use Case:** Useful for queries filtering on multiple fields or requiring sort operations.

□ **Example:**db.collection.createIndex({ field1: 1, field2: -1 });

- The order of fields in a compound index is significant and affects performance.

3.Multikey Indexes :

- **Description:** Indexes on array fields, creating separate index entries for each array element.
- **Use Case:** Ideal for queries on fields that store arrays.
- **Example:**

```
db.collection.createIndex({ arrayField: 1 });
```

4.Text Indexes :

- **Description:** Specialized indexes for text search on string fields.
- **Use Case:** Enables search operations on textual content, supporting stemming, tokenization, and stop words.
- **Example:** `db.collection.createIndex({ field: "text" });`
 - Supports queries like `$text` and operators such as `$search`.

5.Geospatial Indexes:

- **Description:** Indexes for storing and querying geospatial data.
- **Use Case:** Used in applications dealing with location-based data, such as mapping and geo-queries.
- **Types:**
 - **2dsphere:** For spherical data, like Earth coordinates:
`db.collection.createIndex({ location: "2dsphere" });`
 - **2d:** For flat geometry
`db.collection.createIndex({ location: "2d" });`

6.Hashed Indexes:

- **Description:** Indexes that use hash values of fields.
- **Use Case:** Useful for sharded collections to distribute data evenly.
- **Example:** `db.collection.createIndex({ field: "hashed" });`

7.wildcard Indexes:

- **Description:** Indexes on all fields or a subset of fields using wildcard expressions.
- **Use Case:** Flexible indexing for fields with dynamic or unknown names.
- **Example:** `db.collection.createIndex({ "$*": 1 });`

8. Unique Indexes:

- ❑ **Description:** Indexes that enforce unique constraints on indexed fields.
- ❑ **Use Case:** Ensures data integrity by preventing duplicate values.
- ❑ **Example:** `db.collection.createIndex({ field: 1 }, { unique: true });`

9. TTL (Time-to-Live) Indexes;

- ❑ **Description:** Automatically removes documents after a specified time period.
- ❑ **Use Case:** Useful for expiring data like session tokens or logs.
- ❑ **Example:**

`db.collection.createIndex({ createdAt: 1 }, { expireAfterSeconds: 3600 });`

- Documents are removed 3600 seconds after the createdAt timestamp.

10. Partial Indexes:

- ❑ **ase:** Improves efficiency by indexing only relevant documents.
- ❑ **Example:**

`db.collection.createIndex({ field: 1 }, { partialFilterExpression: { status: "active" } });`

11. Sparse Indexes:

- **Description:** Indexes only documents that contain the indexed field, excluding documents where the field is null or missing.
- **Use Case:** Useful for fields that are not present in every document.
- **Example:** `db.collection.createIndex({ field: 1 }, { sparse: true });`

Inserting documents into collection:

```
test> use db
switched to db db
db> db.products.insertMany([
...   { _id: 1, name: "Product A", category: "Electronics", price: 99.99, tags: ["electronics", "gadget"] },
...   { _id: 2, name: "Product B", category: "Clothing", price: 49.99, tags: ["clothing", "fashion"] },
...   { _id: 3, name: "Product C", category: "Electronics", price: 199.99, tags: ["electronics", "gadget"] },
...   { _id: 4, name: "Product D", category: "Books", price: 19.99 }, // No tags
...   { _id: 5, name: "Product E", category: "Electronics", price: 149.99, tags: ["electronics"] }
... ]);
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5 }
}
```

Creating Different Types of Indexes:

```
db> db.products.createIndex({ name: 1 }, { unique: true });
name_1
db> db.products.createIndex({ tags: 1 }, { sparse: true });
tags_1
db> db.products.createIndex({ category: 1, price: -1 });
category_1_price_-1
```

To get the indexes we use the below code:

```
db> db.products.getIndexes();
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true },
  { v: 2, key: { tags: 1 }, name: 'tags_1', sparse: true },
  {
    v: 2,
    key: { category: 1, price: -1 },
    name: 'category_1_price_-1'
  }
]
db> |
```